Student Performance & Attendance Analyzer

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410003	26102000122	BWU/MCA/26/004	Jyoti Boiragi

in partial fulfillment for the award of the degree
of

MASTER OF COMPUTER APPLICATIONS
IN

DEPARTMENT OF COMPUTATIONAL SCIENCES
BRAINWARE UNIVERSITY
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ACKNOWLEDGEMENT

With profound respect and deep gratitude, I express my sincere appreciation to my project supervisor, Mr. Rahul Kumar Ghosh, Assistant Professor, Computational Sciences, Brainware University, for his exceptional guidance, constant supervision, and invaluable suggestions throughout the course of this project. His expertise, constructive feedback, and patience have been instrumental in shaping the direction and successful completion of this work.

I extend my heartfelt thanks to Dr. Jayanta Aich, Head of the Department, Computational Sciences, Brainware University, for providing me with the necessary academic support, computing facilities, and an encouraging environment that enabled me to carry out this project efficiently.

I also thank all the faculty members, technical staff, and my classmates for their direct and indirect support during the development and documentation stages of this project.

Roll No	Registration No	Student Code	Student Name	Signature
26102410003	26102000122	BWU/MCA/26/004	Jyoti Boiragi	

ABSTRACT

This project presents a Python-based application that analyzes student academic performance and attendance data. The system accepts student records, computes key evaluation metrics such as total marks, average marks, percentage, letter grades (A, B, C, D, F), and attendance standing, and automatically identifies students falling below the mandatory 75% attendance threshold. The application incorporates robust exception handling to reject invalid numeric inputs, prevents system crashes, and provides immediate visual feedback.

The project is structured following Object-Oriented Programming (OOP) principles using an encapsulated Student class. Persistent record storage is achieved through Comma-Separated Values (CSV) file handling, allowing data to be saved and reconstructed across sessions. Numerical calculations and cohort statistics (class average, highest percentage, and lowest percentage) are powered by NumPy, while Pandas enables clean tabular data reading. Graphical insights are generated using Matplotlib in three distinct formats: a Bar Chart for individual comparisons, a Line Graph for performance tracking, and a Histogram for score distribution analysis.

A user-friendly Graphical User Interface (GUI) developed in Tkinter provides intuitive controls via labeled entry fields, action buttons, and a formatted display area. The system reduces manual calculation effort for teachers and administrators, eliminates human errors, and demonstrates a practical, end-to-end educational analytics pipeline suitable for college evaluation workflows.

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	1
2	Objective	2
3	Planning	3
4	Requirement Analysis	4
5	System Flow	5
6	Proposed Design	12
7	Experimental Result	13
8	Future Scope	19
9	Conclusion & References	20

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	Context Level Diagram (CLD)	5
5.2	Data Flow Diagram Level 0	6
5.3	Data Flow Diagram Level 1	7
5.4	Use Case Diagram	8
5.5	Activity Diagram	9
5.6	Sequence Diagram	10
5.7	Flow Chart	11
7.1	Data Loading Stage (Main GUI)	13
7.2	Student Record & Performance Analysis	14
7.3	Low Attendance Defaulters Report	15
7.4	Bar Chart Generation	17
7.5	Line Graph Generation	17
7.6	Histogram Distribution Generation	18

LIST OF SYMBOLS & ABBREVIATIONS

Abbreviation	Description
CSV	Comma Separated Values (File format used for student records)
DFD	Data Flow Diagram
CLD	Context Level Diagram
UML	Unified Modeling Language
GUI	Graphical User Interface
OOP	Object-Oriented Programming

CHAPTER 1: INTRODUCTION

Traditional student performance and attendance tracking relies on manual registers, spreadsheets, and time-consuming repetitive calculations. Faculty members and academic coordinators frequently encounter challenges when computing totals, percentages, and grade assignments across multiple subjects for large classrooms. Furthermore, identifying students with poor attendance or borderline academic performance often requires painstaking cross-verification, which increases the likelihood of human calculation errors.

With the rapid adoption of Python and its rich ecosystem of data manipulation and graphical libraries, it is possible to automate these repetitive academic administration tasks. This project focuses on the design and implementation of a desktop application named “Student Performance & Attendance Analyzer”. The system accepts student marks across four core curriculum subjects—Python, DBMS, AI, and JavaScript—alongside class attendance percentages.

The application provides automated grade determination based on standard academic boundaries, identifies low-attendance students falling below the mandatory 75% threshold, ranks students based on performance, and saves structured records into Comma-Separated Values (CSV) files. Built-in integration with NumPy enables fast statistical computations, while Pandas and Matplotlib generate tabular views and clear visual plots (Bar Chart, Line Graph, and Histogram).

A Tkinter-based Graphical User Interface (GUI) provides intuitive buttons and entry fields, making the tool straightforward and accessible for teachers, lab examiners, and academic staff without requiring terminal command execution.

CHAPTER 2: OBJECTIVE

The primary objective of this project is to develop a Python-based desktop application that streamlines the recording, evaluation, and visual analysis of student academic marks and attendance.

Specific Objectives:

1. Data Modeling: To model student records using Object-Oriented Programming (OOP) principles with an encapsulated Student class.

2. Input Validation: To implement defensive programming via try-except blocks, rejecting non-numeric values and out-of-range marks (0–100).

3. Academic Calculations: To automatically compute Total Marks, Mean Average, Percentage, and Letter Grade (A, B, C, D, F) using conditional rules.

4. Attendance Monitoring: To automatically identify and report students whose attendance percentage is strictly below the configured 75% threshold.

5. Performance Analytics: To sort records in descending order and compute cohort metrics (class mean, highest percentage, and lowest percentage) using NumPy.

6. File Persistence: To persist student records in students.csv and reconstruct Student objects dynamically using Pandas.

7. Data Visualization: To generate three distinct Matplotlib visualizations: Bar Chart (comparisons), Line Graph (progression), and Histogram (distribution).

8. User Interface: To provide an intuitive Tkinter desktop interface with labeled entry fields, functional buttons, and an interactive result console.

CHAPTER 3: PLANNING

The planning phase outlines the systematic methodology used to design, organize, and execute the Student Performance & Attendance Analyzer project. Thorough planning ensures modular software architecture, maintainable source code, and complete alignment with the academic syllabus of Problem Solving Using Python (MCA17400).

1. Requirement Identification

The project requirements were derived from real-world classroom assessment workflows. The core requirement is a reliable pipeline: Accept Student Input → Validate Data → Compute Results & Grades → Check Attendance → Save to CSV → Generate Statistical & Graphical Reports.

2. System Design & Tool Selection

Python 3 was selected as the implementation language due to its clarity and extensive standard libraries. Tkinter was chosen for desktop GUI construction, csv for structured file writing, Pandas for tabular data reading, NumPy for vector statistics, and Matplotlib for plotting. A modular architecture separates data modeling, computational logic, GUI interaction, and file operations.

3. Execution Pipeline Structure

• Step 1: Startup verification and auto-loading of existing records from students.csv.

• Step 2: GUI initialization, setting up input entry fields, action buttons, and result text widgets.

• Step 3: Input validation upon user submission (type checking, boundary checks, empty field prevention).

• Step 4: Student object instantiation, automatic total/average/grade calculation, and appending to active student list.

• Step 5: Analytical processing including descending sorting, attendance threshold comparisons, and cohort statistics.

• Step 6: Visual report generation, exporting bar charts, line graphs, and histograms to PNG format.

• Step 7: File persistence, writing updated records to CSV for cross-session continuity.

4. Testing & Validation Plan

The verification strategy includes boundary testing for marks (0, 100, and out-of-range values), non-numeric entry rejection, attendance boundary checks at 74.9% and 75%, file existence checks, and chart generation stability.

CHAPTER 4: REQUIREMENT ANALYSIS

Requirement analysis defines the functional and technical specifications necessary to ensure the application meets academic standards and operates dependably.

1. Functional Requirements

• **Data Input:** The system must accept integer Roll Number, non-empty Name, numeric marks for 4 subjects, and numeric Attendance %.

• **Input Validation:** Non-numeric entries or values outside the 0–100 range must be rejected with informative error dialogs.

• **Calculations:** Automatic computation of total (sum), average/percentage, and letter grade assignment (A: >=80, B: >=70, C: >=60, D: >=50, F: <50).

• **Attendance Alert:** Students with attendance < 75% must trigger an instant warning and appear in the low-attendance report.

• **File Persistence:** Records must be saved to students.csv and reloaded using Pandas DataFrame operations.

• **Visual Analytics:** Creation of three distinct charts: Bar Chart, Line Graph, and Histogram with discrete bins.

2. Software & Hardware Specifications

• **Operating Environment:** Python 3.10 or higher running on Windows, macOS, or Linux.

• **Libraries:** Tkinter (built-in), NumPy (>=1.20), Pandas (>=1.3), Matplotlib (>=3.4).

• **Hardware:** Standard computer system with 2 GB RAM, 100 MB free storage, display resolution of 1024x768 or higher.

CHAPTER 5: SYSTEM FLOW

This chapter presents the architectural diagrams, process models, and workflow representations of the Student Performance & Attendance Analyzer. The diagrams illustrate data movement, entity interactions, decision points, and procedural sequences.

5.1 Context Level Diagram (CLD)

The Context Level Diagram represents the entire application as a single central process interacting with external entities.

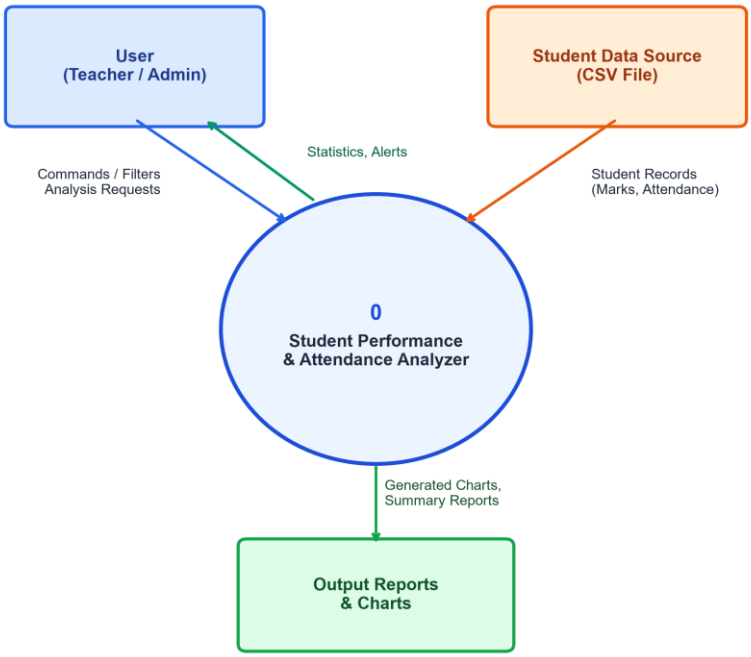


Fig 5.1: Context Level Diagram

5.2 Data Flow Diagram Level 0

The Level 0 DFD illustrates the major functional processes, internal data stores, and high-level data flow paths.

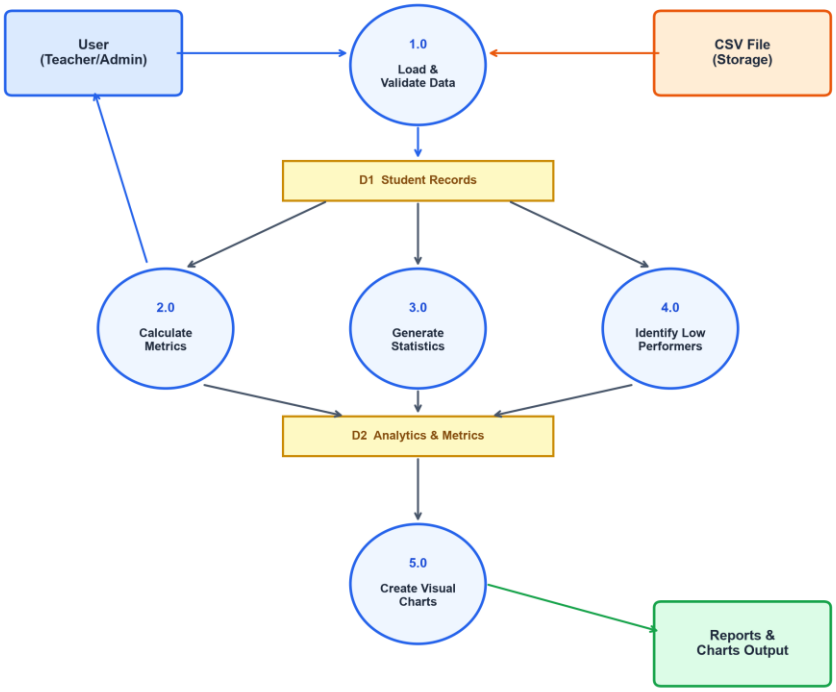


Fig 5.2: Data Flow Diagram Level 0

5.3 Data Flow Diagram Level 1

The Level 1 DFD decomposes the core modules into detailed sub-processes for data loading, metric computation, and report generation.

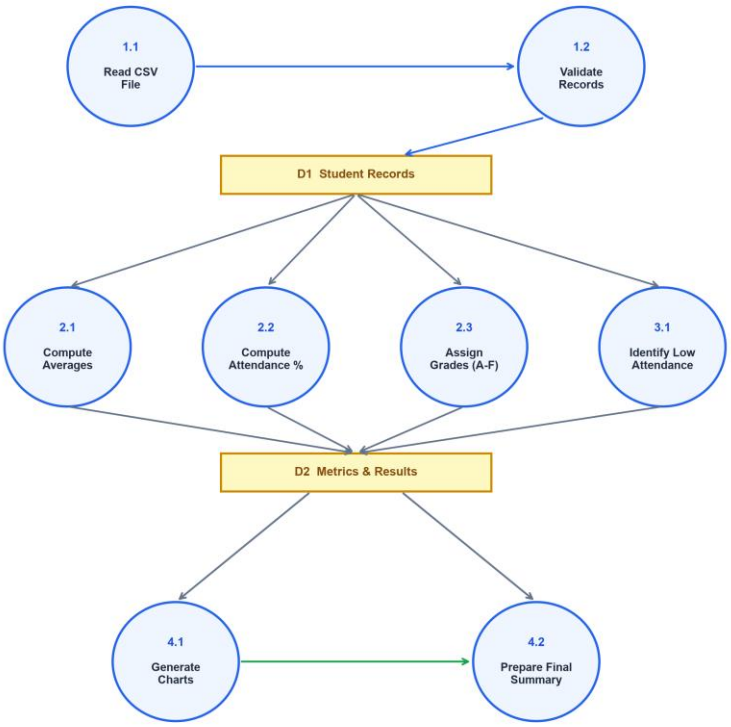


Fig 5.3: Data Flow Diagram Level 1

5.4 Use Case Diagram

The Use Case Diagram depicts the interactions between the primary actor (Teacher/Administrator) and the system's core capabilities.

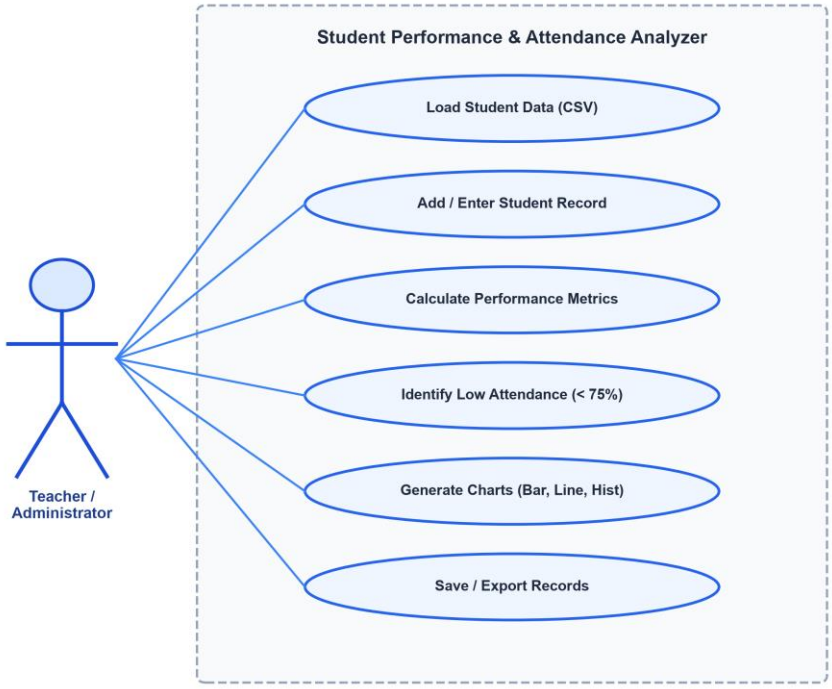


Fig 5.4: Use Case Diagram

5.5 Activity Diagram

The Activity Diagram illustrates the step-by-step procedural workflow, conditional decision branches, and execution paths.

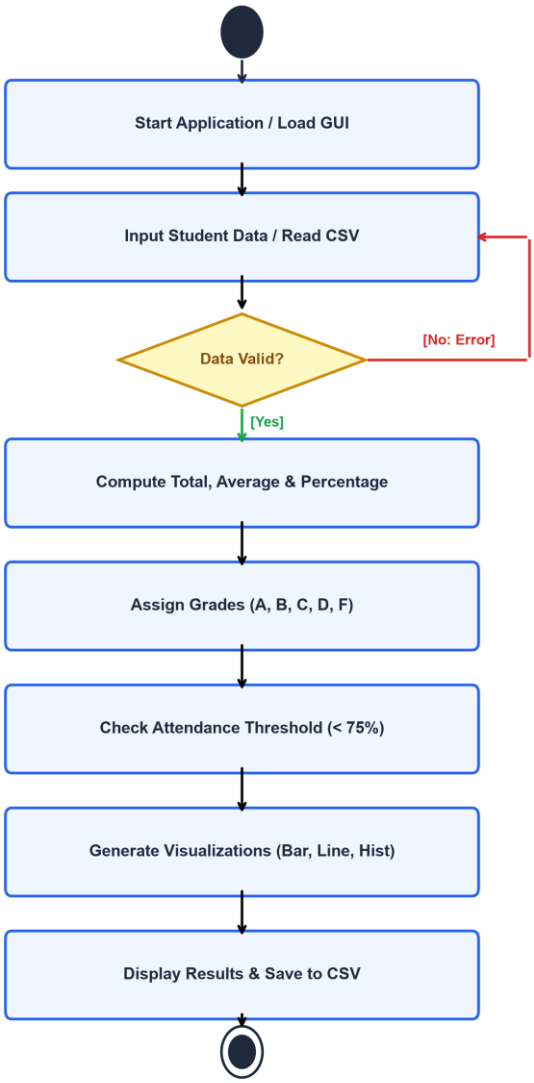


Fig 5.5: Activity Diagram

5.6 Sequence Diagram

The Sequence Diagram traces the chronological message exchanges between the User, GUI Controller, Analysis Engine, and Storage.

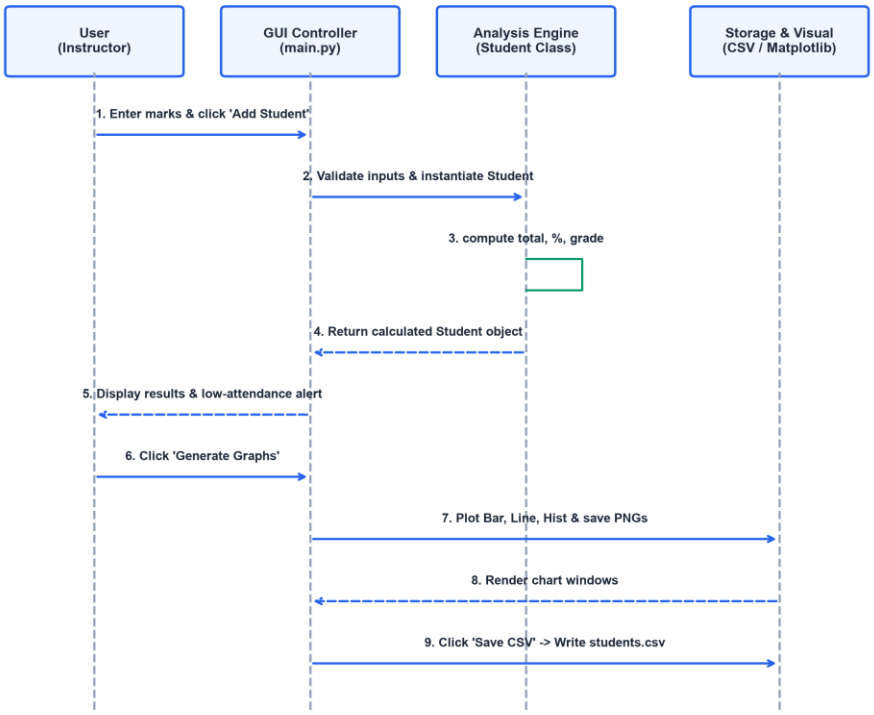


Fig 5.6: Sequence Diagram

5.7 Flow Chart

The Flow Chart details the complete program execution logic using standard flowchart symbols from start to termination.

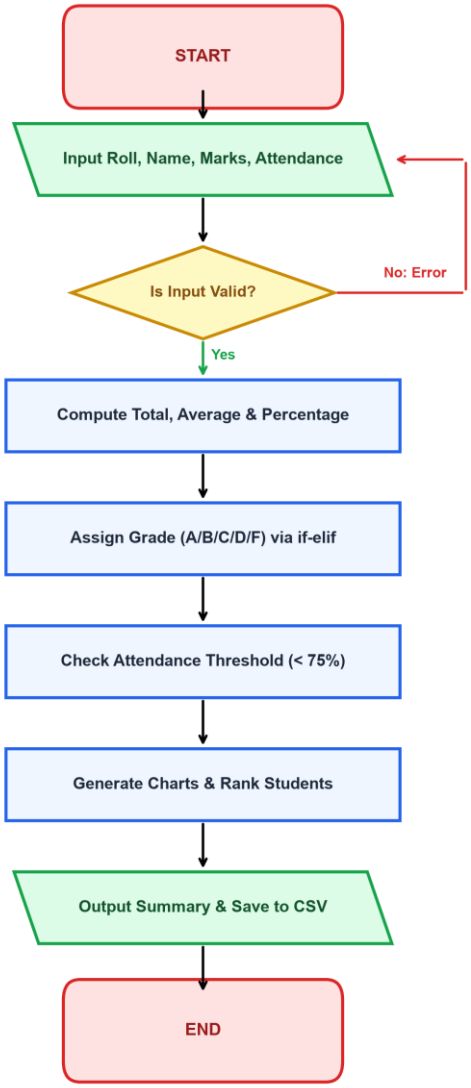


Fig 5.7: Program Flow Chart

CHAPTER 6: PROPOSED DESIGN

The proposed design translates the system requirements into an organized, object-oriented software architecture. The system operates through a sequential pipeline divided into distinct operational stages.

Step 1: Data Loading & Initialization

Upon launch, the application checks for the existence of students.csv using os.path.exists(). If the file is present, Pandas reads the dataset, reconstructs Student objects, and loads them into memory, ensuring previous records are immediately available.

Step 2: Student Data Modeling

The core data structure is modeled using an encapsulated Student class. An instance encapsulates: roll (int), name (str), marks (dict: {subject: score}), attendance (float), total (sum), average (np.mean), percentage, and letter grade.

Step 3: Analytical Logic & Thresholds

• Grade Thresholds: Grade A (>=80%), Grade B (>=70%), Grade C (>=60%), Grade D (>=50%), Grade F (<50%).

• Attendance Compliance: Attendance < 75% triggers an immediate warning flag upon record creation and low-attendance reporting.

• Statistical Analysis: NumPy array processing computes class-level Mean, Highest, and Lowest percentage scores.

• Ranking Engine: Python sorted() function with a lambda key orders students in descending percentage sequence.

Step 4: Visualization Engine

Matplotlib provides visual analytics: Bar Chart (comparing percentages per student), Line Graph (tracking performance trends), and Histogram (grouping scores into discrete ranges: 0–20, 20–40, 40–60, 60–80, 80–100%). Plots are saved as PNG files and shown.

Step 5: File Persistence Architecture

The csv module writes structured headers and student records. The Pandas read_csv() engine reconstructs runtime objects and displays clean formatted tabular output.

CHAPTER 7: EXPERIMENTAL RESULT

The application was tested and evaluated using sample student records. The experimental results verify data loading, input validation, performance ranking, attendance threshold filtering, and chart generation.

1. Data Loading Stage & Main Interface

The main GUI window initializes cleanly with structured input fields, operational buttons, and an interactive display area.

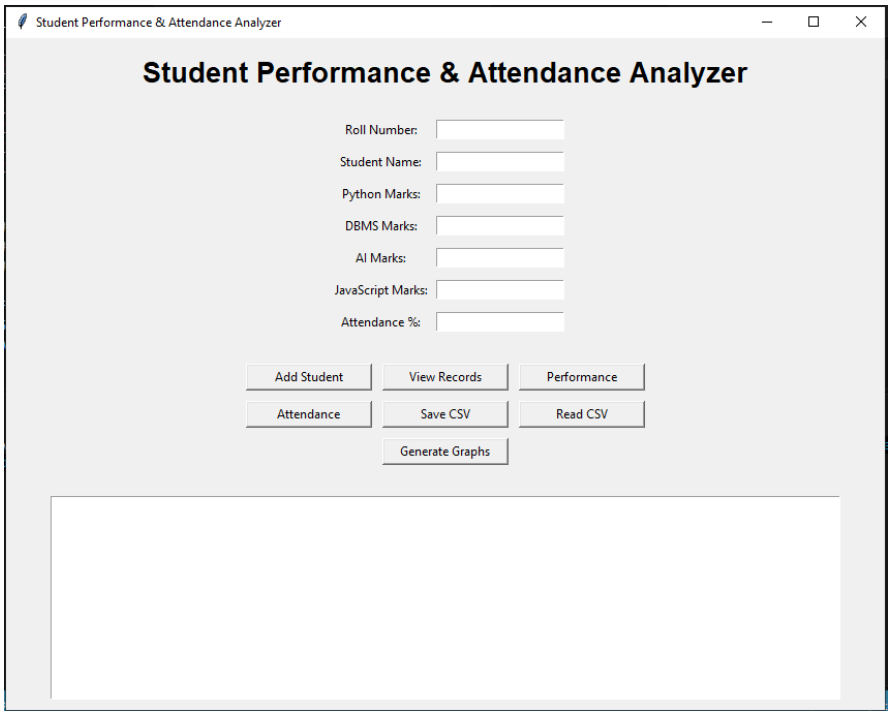


Fig 7.1: Data Loading Stage (Main GUI Window)

2. Student Record & Performance Analysis

Record entry calculates total, percentage, grade, and cohort statistics (Average: 70.58%, Highest: 77.75%, Lowest: 65.25%).

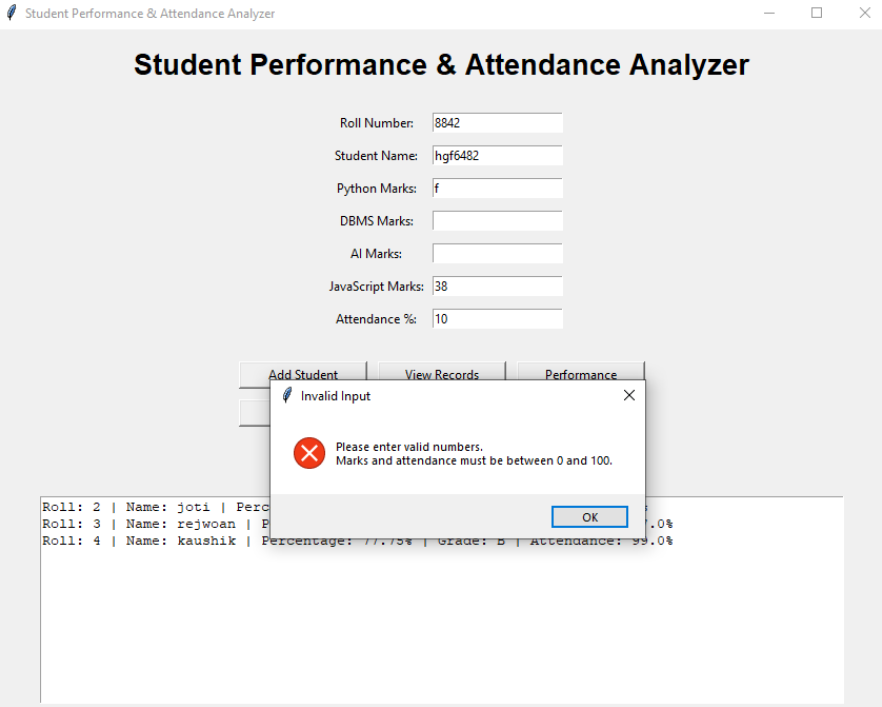


Fig 7.2: Student Record & Performance Analysis

3. Low Attendance Defaulters Report

The attendance analysis identifies and lists students below the 75% threshold (Joti: 55.0%, Rejwoan: 67.0%).

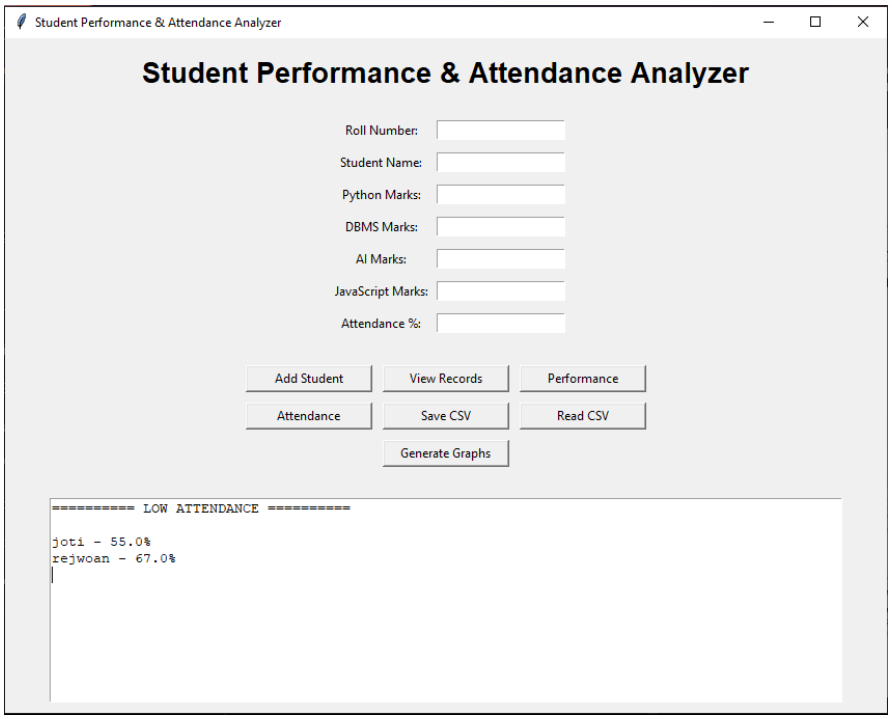


Fig 7.3: Low Attendance Defaulters Report

4. Input Validation & Error Handling

Non-numeric inputs and out-of-range marks trigger an informative error dialog, protecting data integrity.

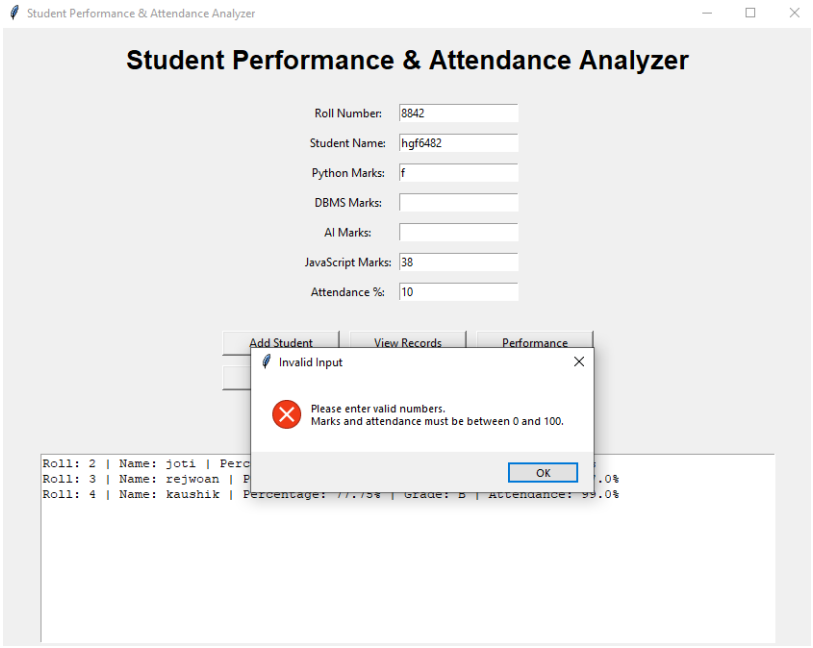


Fig 7.4: Input Validation Error Handling Dialog

5. Graphical Visualizations (Matplotlib)

The system generates three clear visual charts to support academic decision-making:

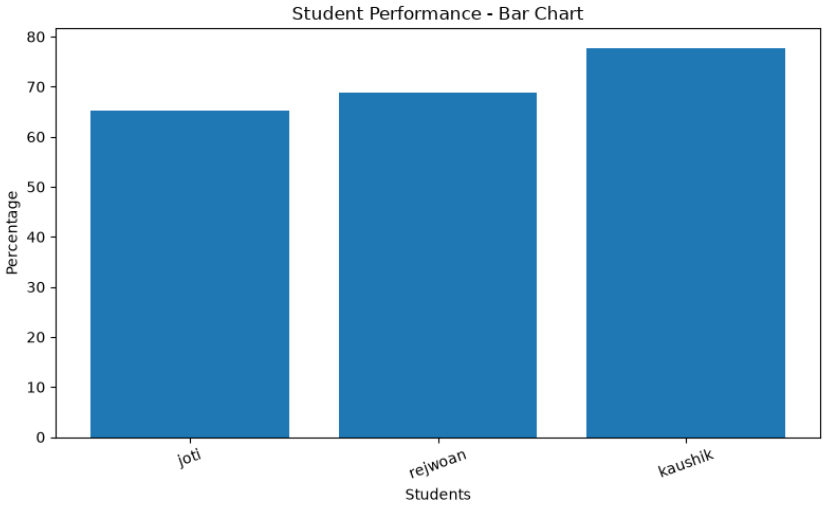


Fig 7.5: Bar Chart Generation (Student Percentage Comparison)

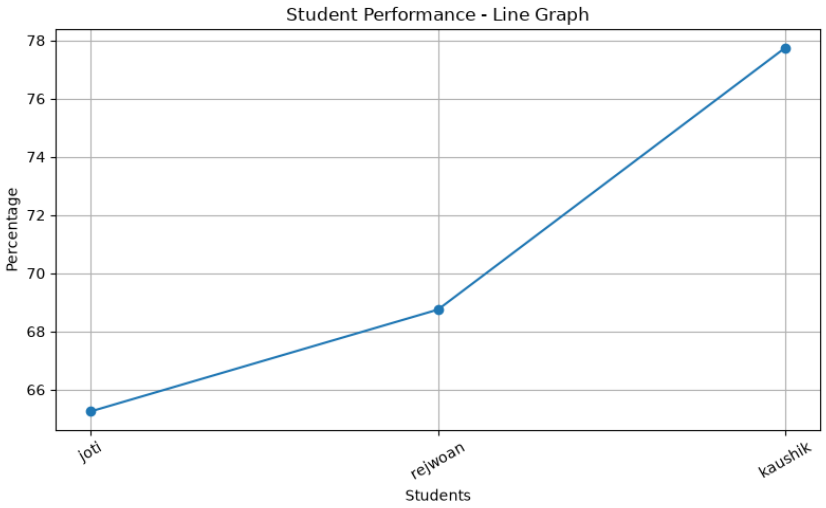


Fig 7.6: Line Graph Generation (Performance Progression)

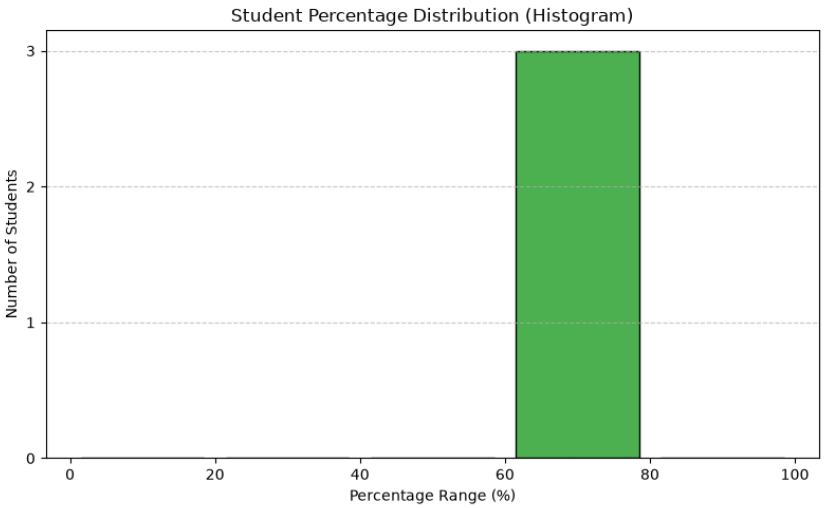


Fig 7.7: Histogram Generation (Score Distribution across 20% bins)

CHAPTER 8: FUTURE SCOPE

While the current implementation fulfills all project objectives, several enhancements can be introduced in future iterations to scale the application for institutional deployment:

1. Database Migration (SQLite / PostgreSQL)

Transitioning from flat CSV files to an embedded relational database (such as SQLite) will support atomic transactions, concurrency, and indexed queries.

2. Dynamic Multi-Subject & Credit Support

Extending the system to allow faculty to define arbitrary subjects, variable maximum marks, and credit-weighted GPA calculations dynamically.

3. Automated PDF Report Card Generation

Adding automated PDF export capabilities to generate individual student grade cards featuring university crests, bar charts, and signatures.

4. Role-Based Authentication & Access Control

Implementing secure login credentials for instructors (full read/write/edit permissions) and students (read-only personal report access).

5. Web-Based Dashboard & Analytics

Porting the desktop interface to a modern web dashboard using Streamlit or Flask for campus-wide accessibility across mobile and desktop devices.

CHAPTER 9: CONCLUSION

The “Student Performance & Attendance Analyzer” project successfully demonstrates an automated, end-to-end software pipeline for academic record management, performance evaluation, and visual analytics. By leveraging Python's core features alongside NumPy, Pandas, and Matplotlib, the application effectively addresses the limitations of manual record keeping.

Key project accomplishments include: robust defensive programming that prevents malformed data entry, automated calculation of totals, percentages, and letter grades, instant identification of low attendance, descending performance ranking, persistent storage via CSV, and automated generation of three distinct chart types. The Tkinter GUI provides an accessible interface that empowers academic personnel to manage and analyze student performance effortlessly.

REFERENCES

1. Python Software Foundation. Python 3 Documentation. Available at: https://docs.python.org/3/

2. McKinney, Wes. Python for Data Analysis (3rd Edition). O'Reilly Media, 2022.

3. The Pandas Development Team. Pandas: Powerful Python Data Analysis Toolkit. Available at: https://pandas.pydata.org/

4. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95.

5. NumPy Developers. NumPy User Guide & Array Computing Reference. Available at: https://numpy.org/doc/

6. Tkinter GUI Programming by Example. Python Standard Library Documentation.

7. Course Material & Syllabus: Problem Solving Using Python (MCA17400), Brainware University, Academic Session 2026–2027.